SOFTWARE REQUIREMENTS SPECIFICATION

StaffDesk

Department & Employee Directory API & Web UI

Phase 1 Capstone Project — Backend Training Program
Client: Crystal Mind — Business Management Solutions

Prepared by	Mazen Adeeb, Engineering Mentor
Prepared for	Ahmed Ghazaly, Junior Backend Developer (Trainee)
Document Version	1.1
Date Issued	31 August 2026
Status	Approved for Development

Document Control

Version	Date	Author	Description
1.0	26 Jul 2026	Mazen Adeeb	Initial issue for Phase 1 capstone assignment.
1.1	31 Aug 2026	Mazen Adeeb	Reissued for Ahmed Ghazaly. Scope widened to a one-week assignment: adds a minimal web UI (Section 6), promotes search & pagination to a requirement, and adds a day-by-day plan (Section 12).

Approval

Role	Name	Signature / Approval
Engineering Mentor	Mazen Adeeb	Approved
Developer (Trainee)	Ahmed Ghazaly	Pending acknowledgment

Table of Contents

1. Introduction 3
2. Project Background 4
3. Functional Requirements — API 5
4. Data Model Requirements 6
5. API Specification 7
6. Functional Requirements — Web UI 8
7. Non-Functional & Process Requirements 9
8. Out of Scope — Phase 1 10
9. Optional Advanced Features (Stretch Goals) 10
10. Deliverables & Acceptance Criteria 11
11. Timeline 11
12. Suggested Day-by-Day Plan 12
Appendix A — Glossary 12
Appendix B — Reference Materials 12

1. Introduction

1.1 Purpose

This document specifies the requirements for **StaffDesk**, a small internal web application — a REST API plus a minimal browser front end — to be designed and implemented by **Ahmed Ghazaly** as the Phase 1 capstone project of the Crystal Mind Backend Training Program. Its purpose is to give a realistic, self-contained brief — the kind a junior developer would receive on a real assignment — that can be satisfied entirely with the material covered in Training Days 1 through 7, and that represents roughly one working week of effort.

1.2 Scope

StaffDesk tracks departments and the employees assigned to them. The backend is intentionally scoped to **read and create** operations only. On top of it sits a deliberately plain web UI — no design work, no framework required — whose only job is to prove the API is usable by a human rather than only by a REST client. Update and delete operations, authentication, automated testing, and deployment are explicitly out of scope for this phase (see Section 8) and will be introduced in later phases of the training program.

1.3 Intended Audience

This document is written for the assigned developer (Ahmed Ghazaly) and his reviewing mentor (Mazen Adeeb). It assumes familiarity with the concepts covered in Training Days 1–7 and does not re-explain them.

1.4 References

Day	Training Document	Relevance to This Project
1	Enterprise Development Fundamentals	REST conventions, request lifecycle, layered thinking
2	Development Workflow	Branching, commits, and pull request process for delivery
3	Environment & Tooling Setup	Local environment used to build and run StaffDesk
4	Git & GitHub Hands-On Lab	Practical Git workflow applied to a real deliverable
5	Databases & SQL Fundamentals	Relational schema design underlying the data model
6	Building Your First API	Express routing, JSON handling, and serving static files
7	Data Modeling & ORMs	Prisma schema, migrations, and relations

2. Project Background

Crystal Mind provides business management software to small and mid-sized companies. Several internal teams have asked for a lightweight way to see which employees belong to which department without opening the full HR system. StaffDesk is commissioned as a small internal tool to answer exactly that need: a simple, reliable API that lists departments, lists employees, and shows who belongs where — and a small internal page that lets a non-technical colleague actually use it.

The API is the substance of the assignment. The UI exists to close the loop: a directory nobody can open in a browser is not a directory. It is expected to be plain HTML with a little JavaScript. No visual design, styling polish, or front-end framework is required or rewarded.

> **FRAMING NOTE FOR THE DEVELOPER**
>
> This is a real-shaped brief on a deliberately small problem. Treat it the way you would a real ticket: read it fully before writing code, note anything ambiguous, and make reasonable, documented decisions on anything this document doesn't pin down. When you make such a decision, record it in the README rather than leaving it implicit in the code.

2.1 Effort Expectation

This brief is sized at approximately **five working days** for a developer who has completed Days 1–7. Section 12 suggests one way to distribute that work. The plan is a guide, not a contract — what is assessed is the finished deliverable in Section 10, not adherence to the schedule.

3. Functional Requirements — API

Each requirement below is tagged **MUST** (required for acceptance) or **SHOULD** (expected, minor deductions if skipped). Priority follows the MoSCoW convention used across Crystal Mind engineering.

ID	Requirement	Priority
FR-1	The system shall expose all API endpoints under a versioned base path, `/v1`.	**MUST**
FR-2	The system shall define a **Department** resource with a unique identifier, name, and location.	**MUST**
FR-3	The system shall define an **Employee** resource with a unique identifier, full name, job title, and a reference to exactly one Department.	**MUST**
FR-4	`POST /v1/departments` shall create a new department from a JSON body and return it with a **201** status.	**MUST**
FR-5	`GET /v1/departments` shall return every department as a JSON array.	**MUST**
FR-6	`POST /v1/employees` shall create a new employee attached to an existing department, returning the created employee with a **201** status. Attempting to attach an employee to a department that does not exist shall fail with a **400** status rather than silently succeeding.	**MUST**
FR-7	`GET /v1/employees` shall return every employee as a JSON array, with each employee record including its department's *name* (not just the raw department id).	**MUST**
FR-8	`GET /v1/departments/:id/employees` shall return only the employees belonging to the specified department, as a nested resource.	**MUST**
FR-9	`GET /v1/employees` shall accept an optional `search` query parameter performing a case-insensitive partial match against `fullName`, returning only matching employees.	**MUST**
FR-10	`GET /v1/employees` shall accept optional `page` and `limit` query parameters for page-based pagination, with documented defaults. Search and pagination shall work together.	**MUST**
FR-11	`GET /v1/departments/:id` shall return a single department, including a count of the employees assigned to it.	**SHOULD**
FR-12	All request and response bodies shall be JSON, following the resource-and-verb conventions covered in Day 1.	**MUST**
FR-13	Requesting a department by an id that does not exist shall return a **404** status rather than an empty **200** or a server error.	**SHOULD**
FR-14	Creating a department or employee with a missing or empty required field shall return a **400** status with a short JSON message naming the problem. (Basic checks only — a validation library is out of scope, see Section 8.)	**SHOULD**

4. Data Model Requirements

StaffDesk's data model is a single one-to-many relationship: one **Department** has many **Employees**, and every Employee belongs to exactly one Department — the same shape practiced in Day 5 (SQL) and Day 7 (Prisma modeling and migrations). The schema below is a specification of the required fields, not a prescription of exact Prisma syntax; naming may be adapted to the codebase's conventions as long as the relationship and constraints hold.

Department

Field	Specification
id	Integer, primary key, auto-generated
name	String, required, e.g. “Engineering”
location	String, required, e.g. “Cairo Office”
employees	One-to-many relation to Employee

Employee

Field	Specification
id	Integer, primary key, auto-generated
fullName	String, required
jobTitle	String, required
departmentId	Integer, foreign key referencing Department.id, required

> **DATA INTEGRITY EXPECTATION**
> An Employee row must never reference a Department that doesn't exist. This should be enforced by the foreign key relationship itself (Day 7), not just by application-level convention.

4.1 Seed Data

The repository shall include a seed script that populates at least **4 departments** and **25 employees**. Twenty-five rows is deliberate: it is enough that pagination (FR-10) and search (FR-9) do something visible instead of fitting on one screen. The seed must be re-runnable from a clean database as a documented README step.

5. API Specification

Method	Endpoint	Description	Success
POST	`/v1/departments`	Create a department. Body: { name, location }	**201**
GET	`/v1/departments`	List all departments.	**200**
GET	`/v1/departments/:id`	Fetch one department, with its employee count.	**200**
GET	`/v1/departments/:id/employees`	List employees belonging to one department.	**200**
POST	`/v1/employees`	Create an employee. Body: { fullName, jobTitle, departmentId }	**201**
GET	`/v1/employees`	List employees, each including its department name. Optional query: search, page, limit.	**200**

Exact response field naming (camelCase vs. snake_case, nesting of the department object vs. a flat `departmentName` field) is left to the developer's judgment, consistent with Section 2's framing note — document the choice in the project README.

5.1 Paginated Response Shape

A paginated list must tell the caller enough to render page controls: at minimum the current page, the page size, and the total number of matching records. The exact envelope is the developer's choice, but it must be consistent and documented. For example:

```
GET /v1/employees?search=ali&page=2&limit=10

{
  "data": [ { "id": 11, "fullName": "...", "jobTitle": "...",
         "departmentName": "Engineering" }, ... ],
  "page": 2,
  "limit": 10,
  "total": 37
}
```

Out-of-range values (`page=0`, a negative `limit`, a non-numeric value) must not crash the server. Decide on sane behaviour — clamp to a default, or reject with a 400 — and document which you chose.

6. Functional Requirements — Web UI

StaffDesk shall ship a minimal browser interface served by the same Express application (Day 6 covers serving static files). Its purpose is to demonstrate that the API in Section 5 is complete and usable end-to-end by a person. It is a functional requirement, not a design exercise.

> **EXPLICITLY NOT REQUIRED**
> No visual design, brand styling, responsive layout, animation, accessibility audit, or front-end framework (React, Vue, Angular) is required. Plain HTML, a single stylesheet if you want one, and vanilla JavaScript using fetch() is the expected and fully acceptable solution. A UI that looks unstyled but works correctly scores higher than a styled one that mis-handles an error.

ID	Requirement	Priority
UI-1	The web UI shall be served by the StaffDesk Express application itself — no separate build step, dev server, or second process.	**MUST**
UI-2	The UI shall consume the Section 5 API over HTTP. No page may read from the database directly or embed server-rendered data as a substitute for calling the API.	**MUST**
UI-3	A **Departments** view shall list every department with its name and location, fetched from GET /v1/departments.	**MUST**
UI-4	An **Employees** view shall list employees in a table showing full name, job title, and department name.	**MUST**
UI-5	The Employees view shall provide a search box that filters the list by name, driven by the API's search parameter (FR-9) — not by filtering an already-downloaded array in the browser.	**MUST**
UI-6	The Employees view shall provide page controls (previous / next, or numbered pages) driven by the API's page and limit parameters (FR-10), and shall show which page of how many is being viewed.	**MUST**
UI-7	Selecting a department shall show only that department's employees, using GET /v1/departments/:id/employees.	**MUST**
UI-8	A form shall allow creating a new department. On success the department list shall reflect the new record without a manual page refresh.	**MUST**
UI-9	A form shall allow creating a new employee, with the department chosen from a dropdown populated from the live department list — not a free-text id field.	**MUST**
UI-10	When the API returns an error (e.g. the 400 of FR-6 or FR-14), the UI shall display a readable message to the user rather than failing silently or showing a raw stack trace.	**MUST**
UI-11	While a request is in flight, the UI shall show some indication that it is loading, and an empty result set shall render an explicit “no results” message rather than a blank area.	**SHOULD**
UI-12	The UI shall be usable without reading the source: every control shall be labelled in plain language.	**SHOULD**

6.1 Suggested Structure

A single page with two sections, or two small pages linked to each other, are both acceptable. Anything that satisfies UI-1 through UI-12 is acceptable. Record the structure you chose, and why, in the README.

7. Non-Functional & Process Requirements

ID	Requirement
NFR-1	Built with Node.js and Express, per the stack established in Day 6.
NFR-2	Data persisted in PostgreSQL, accessed through Prisma, with all schema changes captured as committed migrations (Day 7).
NFR-3	The project must run locally from a clean clone using only the setup steps documented in the project's own README, consistent with the environment established in Day 3. This includes running the seed script of Section 4.1.
NFR-4	All work shall be committed to a Git repository using feature branches and small, meaningful commits; the finished project shall be delivered as a reviewed pull request into `main`, per Day 2 and Day 4's workflow — even though this is a solo project, follow the same discipline as if a teammate were reviewing.
NFR-5	Endpoint naming and structure shall follow REST conventions (plural resource nouns, correct verb-to-operation mapping) as covered in Day 1.
NFR-6	Route handlers shall not contain database queries inline. Keep a visible separation between routing, business logic, and data access — the layered thinking of Day 1. Exact folder structure is the developer's choice.
NFR-7	Database credentials and any other environment-specific configuration shall be read from environment variables and must not be committed. Provide a `.env.example` listing the required variables.
NFR-8	The server must not crash on a malformed request. An unexpected failure shall return a 500 with a JSON body, and the process shall stay alive.

8. Out of Scope — Phase 1

The following are deliberately excluded from this phase and **should not** be implemented yet. They map to material later in the training program and are listed here so their absence is a design decision, not an oversight:

- **Authentication & authorization** — introduced in Day 9. StaffDesk's endpoints and UI are open in this phase.
- **Update (PUT/PATCH) and delete operations** — introduced alongside full CRUD in Day 8. The UI therefore has no edit or delete controls.
- **Formal input validation & centralized error handling** — introduced in Day 11. Basic, sensible handling (FR-6, FR-13, FR-14, NFR-8) is still expected.
- **Automated tests** — introduced in Day 10. Manual verification via a REST client and the UI is sufficient for this phase.
- **Front-end frameworks and build tooling** — outside the backend track entirely. See the callout in Section 6.
- **Deployment** to any hosted or production environment.

9. Optional Advanced Features (Stretch Goals — Not Required)

> **OPTIONAL — DOES NOT AFFECT ACCEPTANCE**
> A submission that meets Sections 3–7 in full is considered complete and acceptable without any of the following. Attempting one is encouraged as a way to explore slightly beyond the current curriculum using only tools already introduced.

- **Sorting.** Let GET /v1/employees accept a `sort` parameter (e.g. by full name or job title, ascending or descending), with matching column headers in the UI.
- **Filter by department in the query string.** Add an optional `departmentId` parameter to GET /v1/employees, combinable with search and pagination.
- **Directory summary.** An endpoint returning each department with its employee headcount, rendered as a small summary line at the top of the UI.
- **Deep-linkable UI state.** Reflect the current search term and page in the browser's URL so a filtered view can be copied and shared.

10. Deliverables & Acceptance Criteria

10.1 Deliverables

- A GitHub repository containing the StaffDesk source code, including the web UI.
- A README with local setup instructions (including database setup and the seed script), a list of available endpoints, and a short record of the judgment calls made where this document left a choice open.
- A `.env.example` file listing every required environment variable, per NFR-7.
- A pull request merging the completed work into `main`, per NFR-4.
- A short written note confirming manual verification of every endpoint in Section 5 and every requirement in Section 6 (screenshots or a written walkthrough are both acceptable).

10.2 Acceptance Criteria

Ref.	Acceptance Check
FR-1–3	Both resources exist with the fields specified in Section 4, and the relationship is enforced at the database level.
FR-4–8	Every endpoint in Section 5 behaves exactly as specified when exercised manually.
FR-9–10	Search and pagination work independently and in combination, against the seed data of Section 4.1, and out-of-range parameters do not crash the server.
FR-11–14	Responses are valid JSON with correct, deliberate status codes — not default framework behavior left unexamined.
UI-1–12	The UI is reachable from the running server, exercises the API for every listed behaviour, and surfaces errors and empty states to the user.
NFR-1–8	The project runs from a clean clone via the README alone, its Git history shows a real feature-branch-to-PR workflow, and no secrets are committed.

11. Timeline

StaffDesk is the capstone checkpoint for Phase 1 of the training program and is assigned upon completion of Training Day 7. It should be completed before starting Day 8 (CRUD APIs End-to-End), which builds directly on these same skills. The brief is sized at approximately one working week (see Section 2.1).

There is no fixed deadline in this document — agree a target date directly with your mentor. Raise blockers on the day you hit them rather than at the end of the week.

12. Suggested Day-by-Day Plan

This plan is advisory. It exists so the week has a visible shape and so a slipping day is noticed early. Nothing here is assessed — Section 10 is.

Day	Focus	Done when…
1	Project setup and data model. Repository, Express skeleton, Prisma schema, first migration, seed script.	The seed script populates a clean database with the Section 4.1 data.
2	Core read and create endpoints: FR-4 through FR-8, with the layered structure of NFR-6.	Every Section 5 endpoint returns correct data in a REST client.
3	Search, pagination, single-department fetch, and status-code correctness: FR-9 through FR-14.	Search and pagination combine correctly; bad input returns a deliberate status code.
4	Web UI: department and employee lists, department drill-down, search box and page controls (UI-1 to UI-7).	A colleague could browse the directory without being told how.
5	UI forms, error and loading states (UI-8 to UI-12), README, manual verification note, and the pull request.	The project runs from a clean clone via the README alone and the PR is open.

Appendix A — Glossary

Resource
A noun-based entity exposed by a REST API (e.g. a department, an employee).

Nested Resource
A resource scoped under a parent in the URL, e.g. /departments/:id/employees.

Migration
A version-controlled, ordered description of a schema change, applied via Prisma.

Foreign Key
A column referencing another table's primary key, used to model the Department–Employee relationship.

Pagination
Returning a large result set in fixed-size pages rather than all at once, via skip/take in Prisma.

MoSCoW Priority
Must / Should / Could / Won't — a simple way of ranking requirement priority; this document uses Must and Should.

Appendix B — Reference Materials

See Section 1.4 for the mapping between this project's requirements and Training Days 1–7. All seven training documents remain the authoritative reference for concepts used in this specification and are not reproduced here.